# Convolutional Neural Networks

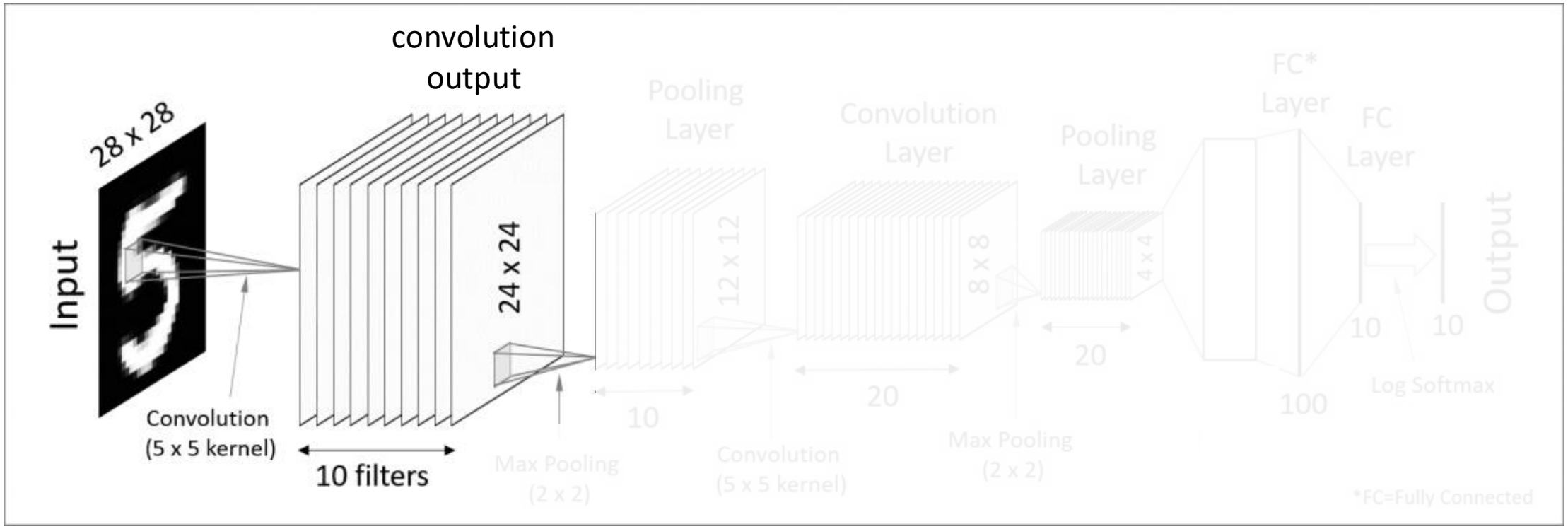
**Erisa Terolli**

Slide Credits to Shusen Wang

# CNN Structure

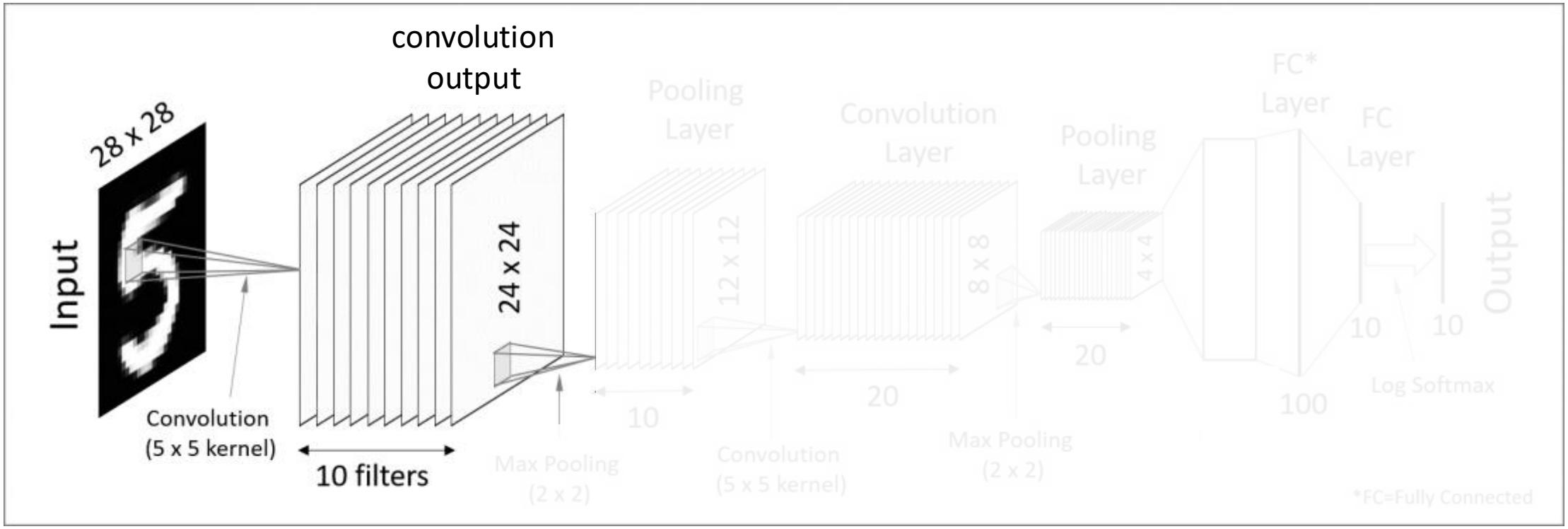
# CNN: Convolutional Layer 1

- Use multiple filters (kernels) for the feature mapping. E.g.,
  - using ten $5 \times 5$ filters, (merely $10 \times 5 \times 5 = 250$ parameters!)
  - the results are ten $24 \times 24$ matrices.



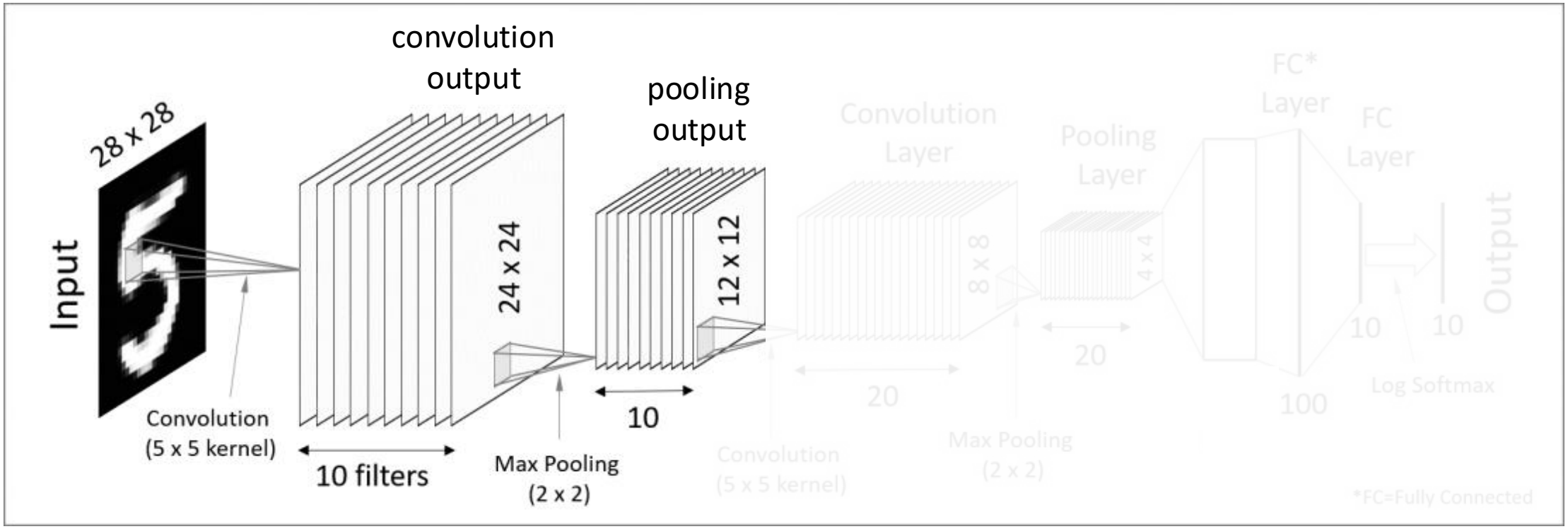
# CNN: Convolutional Layer 1

- Use multiple filters (kernels) for the feature mapping.
- Apply an activation function (e.g., ReLU).
- A Convolutional layer = convolution + activation.



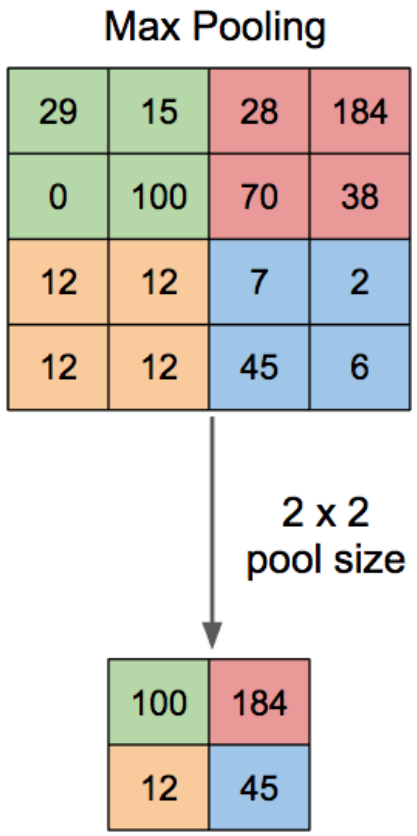
# CNN: Pooling Layer 1

- Pooling reduces the dimensionality of each feature map.
- E.g., Max Pooling, Average Pooling, etc.



# Pooling

- Pooling reduces the dimensionality of each feature map.
- E.g., Max Pooling, Average Pooling, etc.

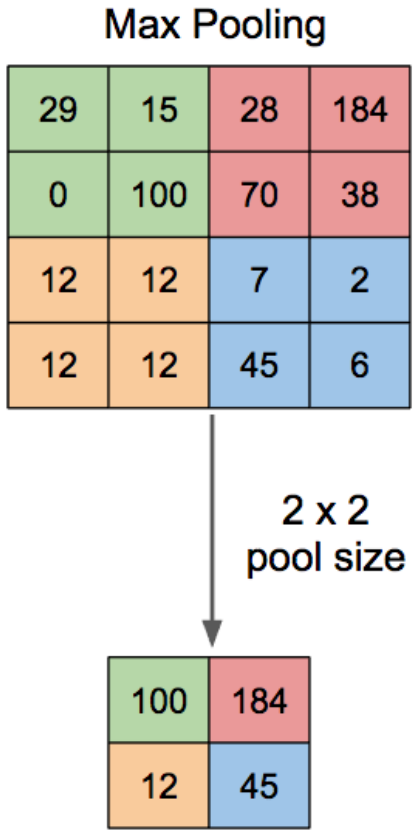



**Settings:**

- Max Pooling.
- Pool size $= 2 \times 2$.
- No overlap (stride $= 2 \times 2$).

# Pooling

- Pooling reduces the dimensionality of each feature map.
- E.g., Max Pooling, Average Pooling, etc.



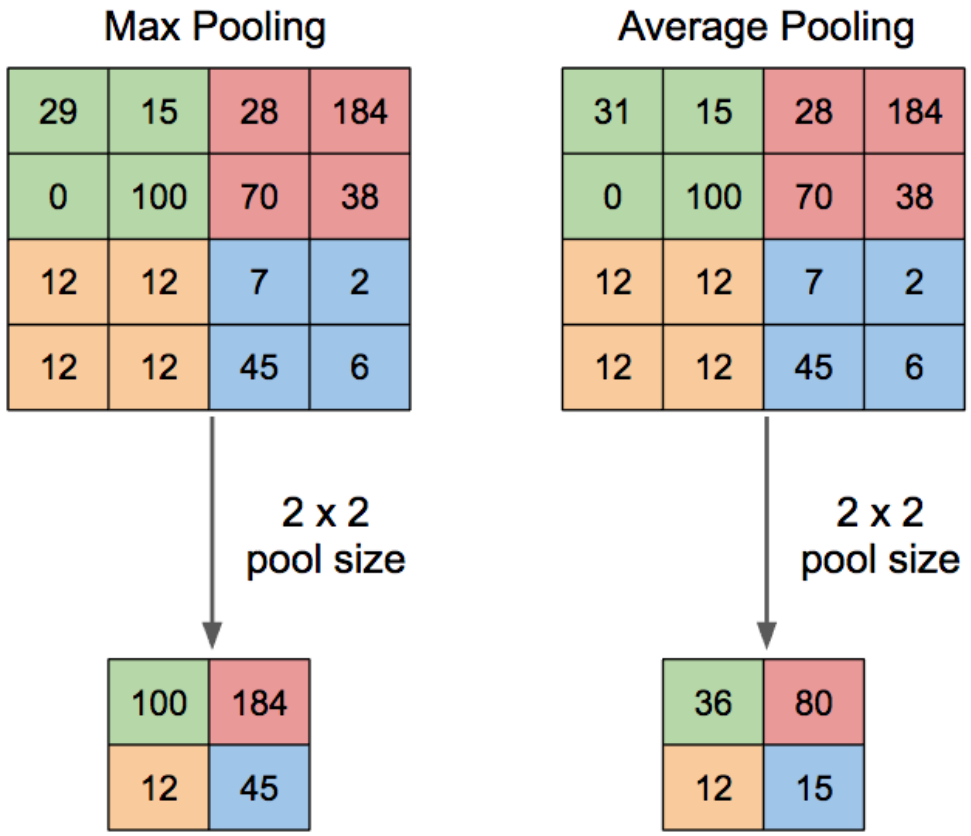

**Settings:**

- Max Pooling.
- Pool size $= 2 \times 2$.
- No overlap (stride $= 2 \times 2$).

**Question:** What if the stride is $1 \times 1$?

# Pooling

- Pooling reduces the dimensionality of each feature map.
- E.g., Max Pooling, Average Pooling, etc.

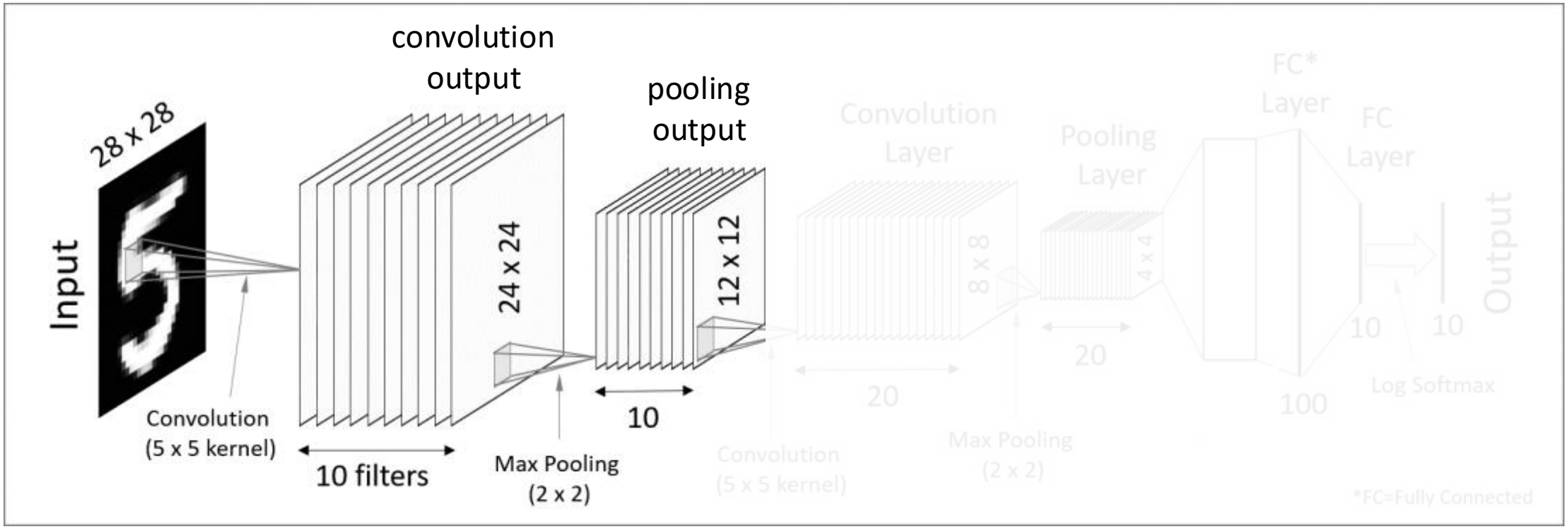



**Settings:**

- Average Pooling.
- Pool size $= 2 \times 2$.
- No overlap (stride $= 2 \times 2$).

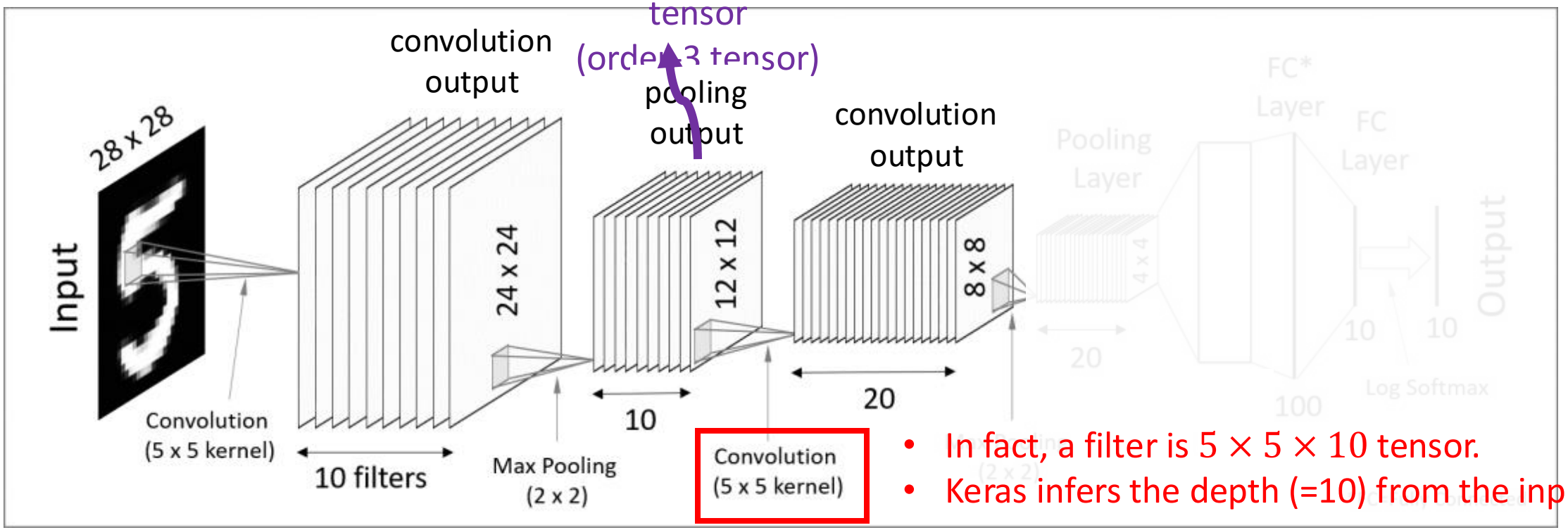
# CNN: Pooling Layer 1

- Pooling reduces the dimensionality of each feature map.
- E.g., Max Pooling, Average Pooling, etc.

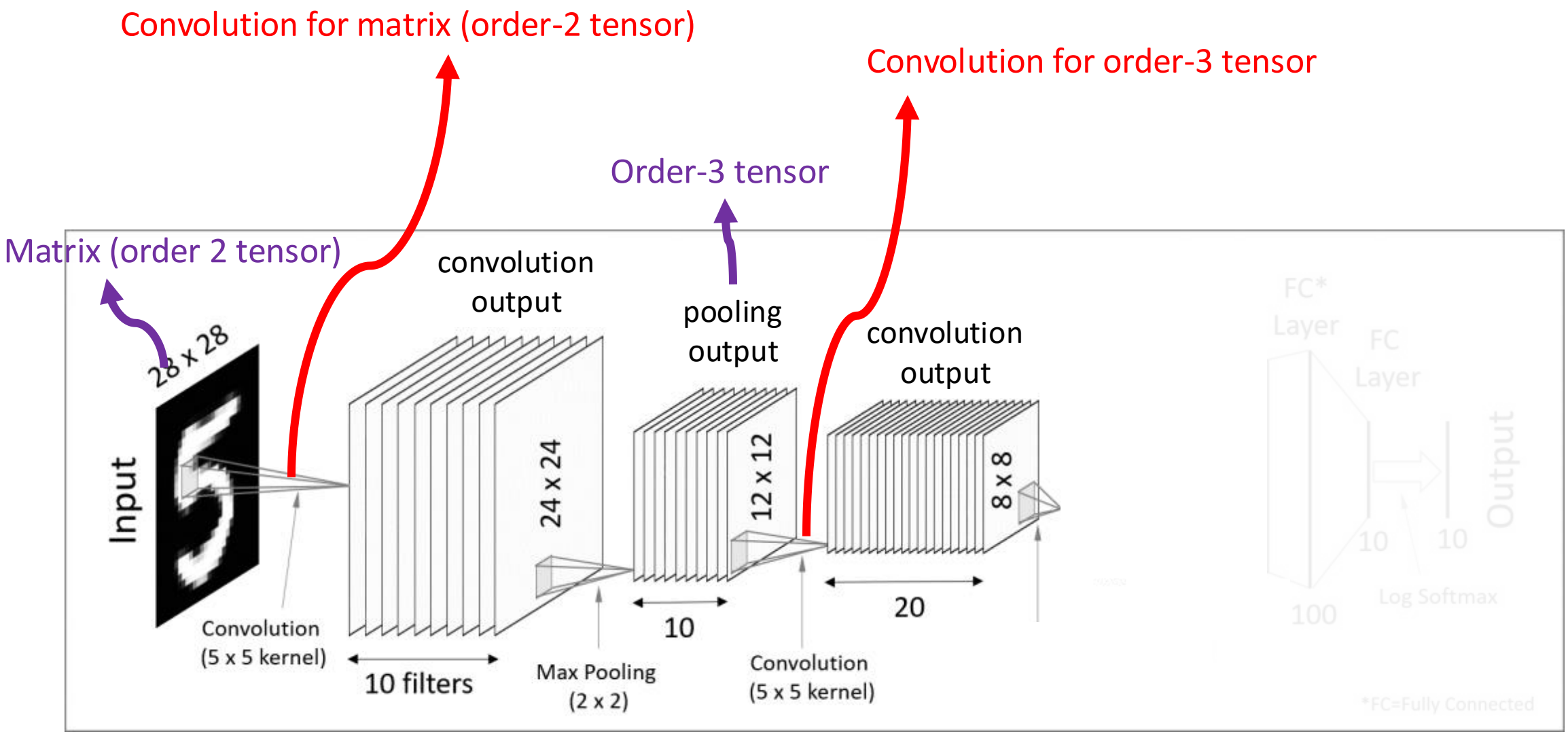


# CNN: Convolutional Layer 2

- Perform tensor convolution.
- Apply $5 \times 5 \times 10$ tensor filter to the $12 \times 12 \times 10$ tensor input.
- Twenty such filters ➔ twenty $8 \times 8$ feature maps.

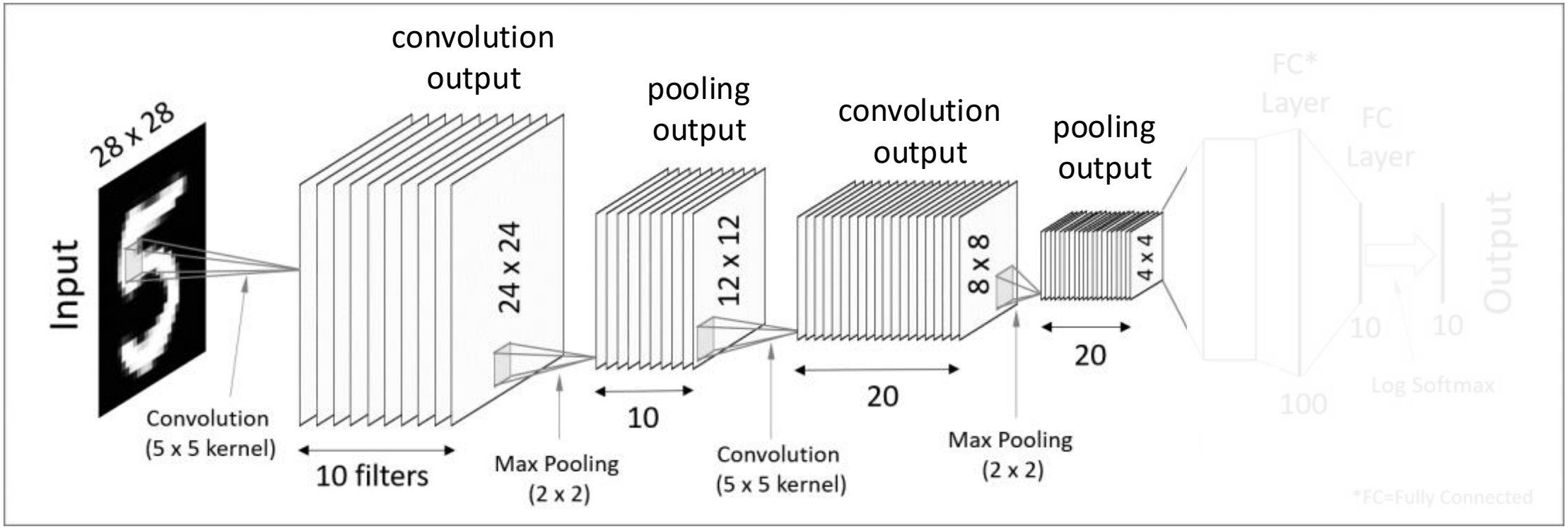


# CNN: Convolutional Layers



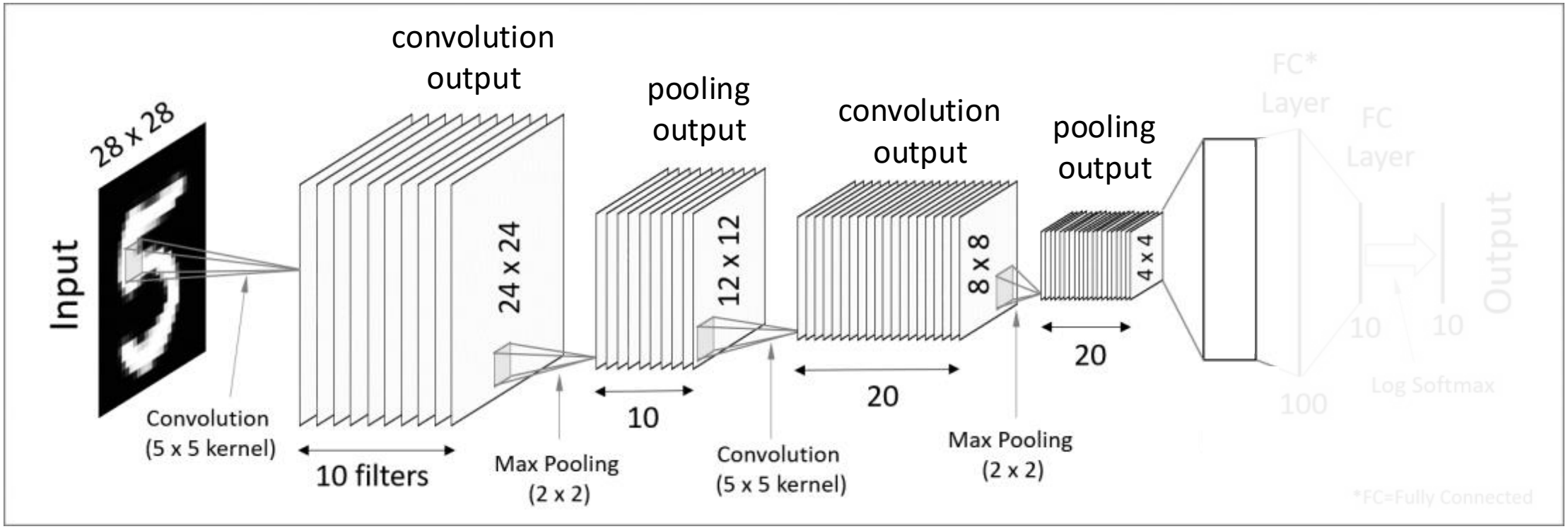
# CNN: Pooling Layer 2

- $2 \times 2$ Max Pooling applied to an $8 \times 8$ matrix ➔ a $4 \times 4$ matrix.



# CNN: Flatten (Vectorization)

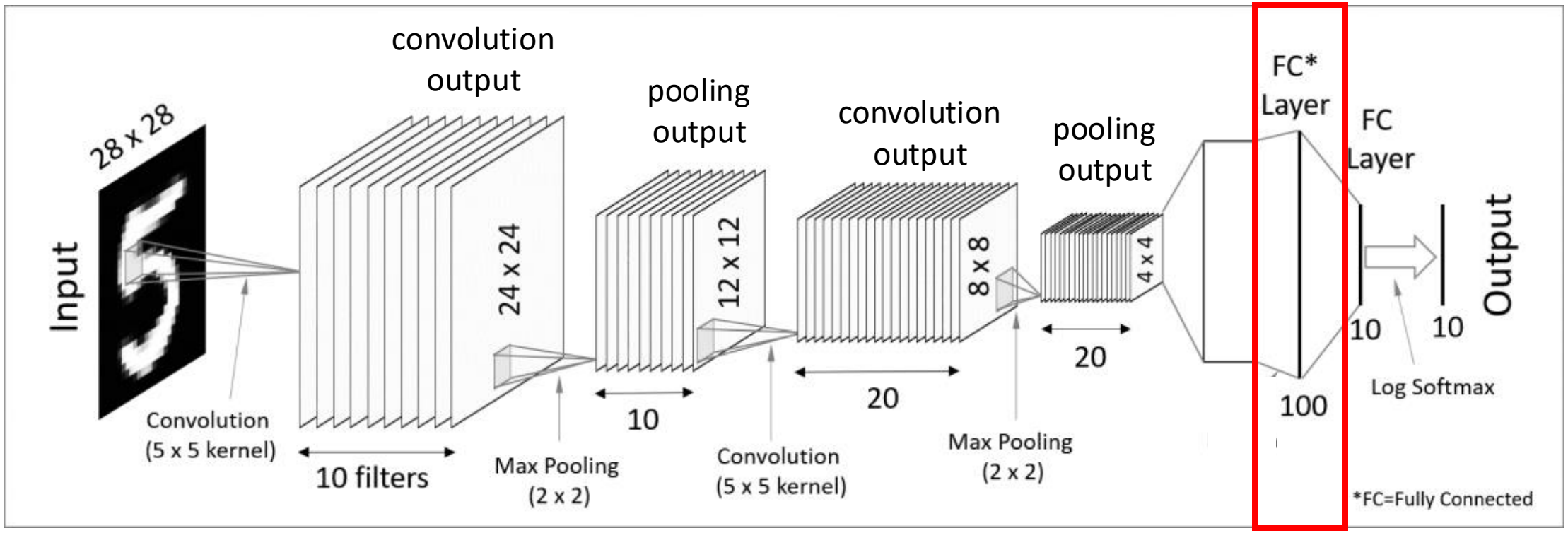
- Flatten the $4 \times 4 \times 20$ tensor.
- Get a $320$-dim vector.



# CNN: Fully-Connected Layers

- Add one or more fully-connected layers

This one is optional.



# CNN: How Many Trainable Parameters?

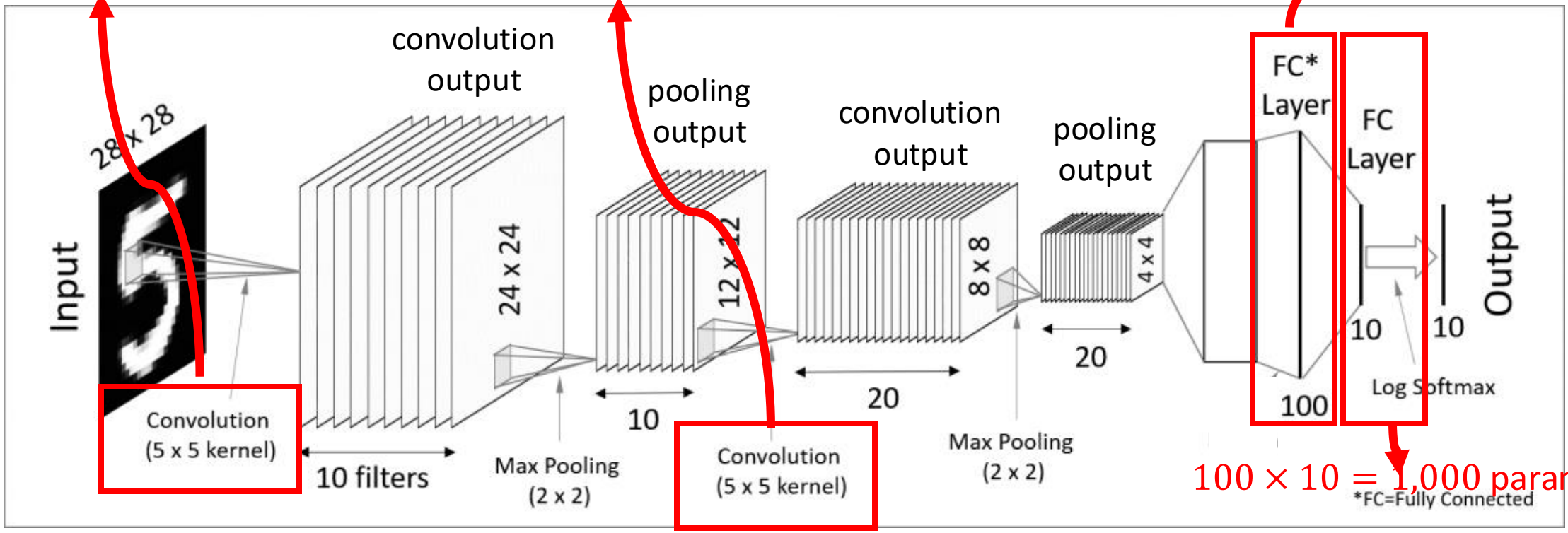
$10 \times 5 \times 5 = 250$ parameters

- 10: number of filters
- $5 \times 5$: size of filters

$20 \times 5 \times 5 \times 10 = 5000$ parameters

- 20: number of filters
- $5 \times 5 \times 10$: size of filters

$320 \times 100 = 32{,}000$ parameters




$100 \times 10 = 1{,}000$ parameters

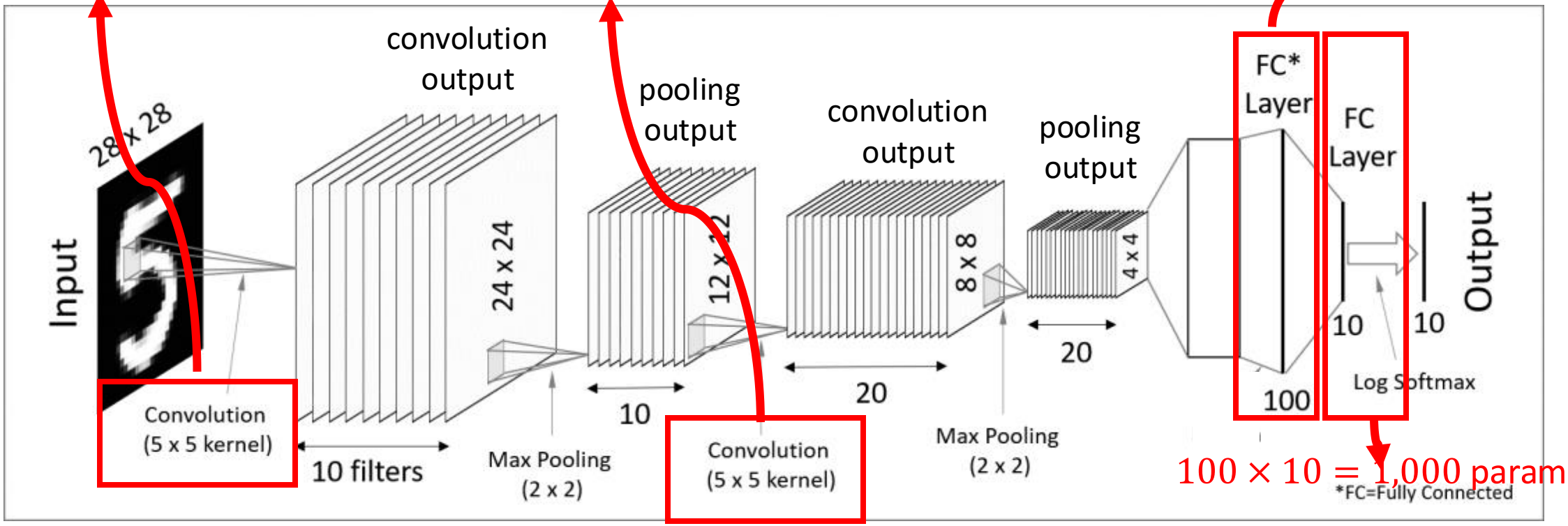
# CNN: How Many Trainable Parameters?

**Answer**: $> 38{,}250$ parameters (not counting the intercepts).

$10 \times 5 \times 5 = 250$ parameters
- 10: number of filters
- $5 \times 5$: size of filters

$20 \times 5 \times 5 \times 10 = 5000$ parameters
- 20: number of filters
- $5 \times 5$: size of filters
- 10: depth of filters

$320 \times 100 = 32{,}000$ parameters

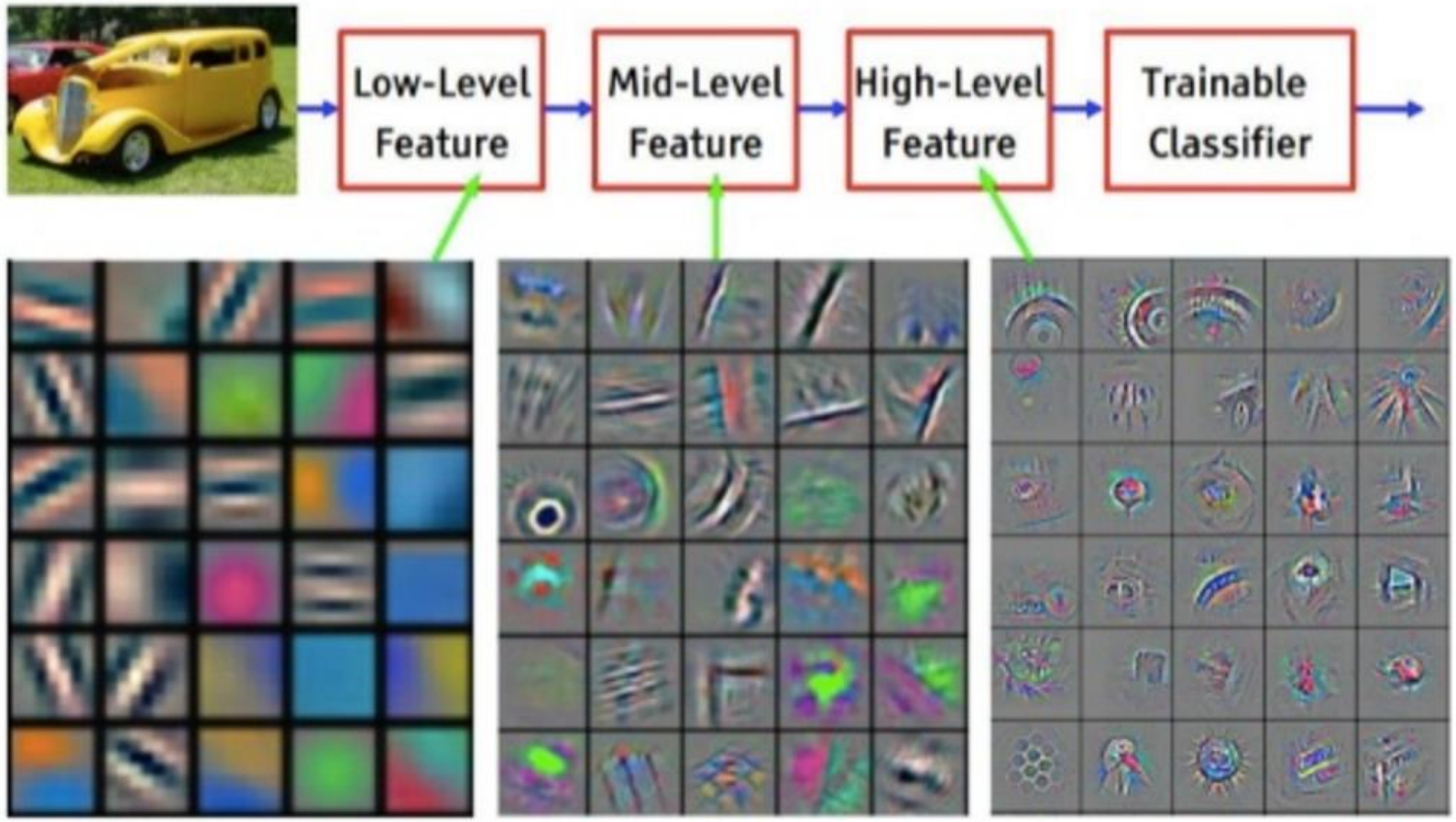



$100 \times 10 = 1{,}000$ parameters

# CNN: Re-Cap

**Question**: Why convolutional layers?

**Answer**: detect features like edge, shape, pattern, etc.



# CNN: Re-Cap

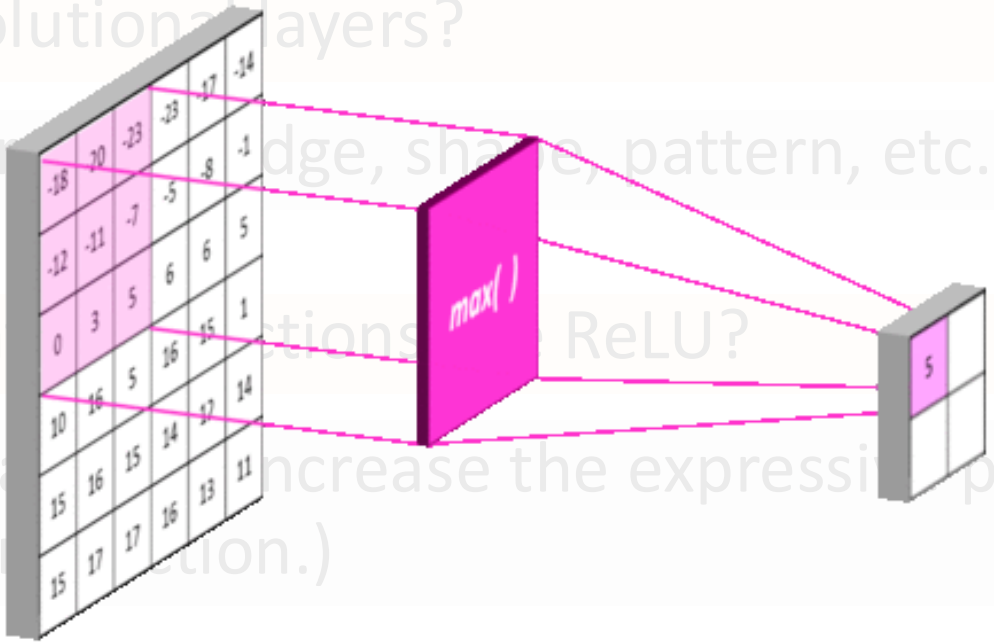
**Question**: Why convolutional layers?

**Answer**: detect features like edge, shape, pattern, etc.

**Question**: Why activation functions like ReLU?

**Answer**: Add **nonlinearity** to increase the expressive power. (Note that convolution is a **linear** function.)

# CNN: Re-Cap




**Question**: Why pooling?

**Answer**: Drastically reduce the shape of features.

# Hyper-Parameters of CNN

# Structure Hyper-Parameters

- Convolutional layers
  - # Filters
  - Filter shape
  - Stride
  - Zero-padding
- Pooling layers
  - MaxPool or AvgPool
  - Pool size
  - Pool stride
- Fully-connected layers
  - Width
- Activation functions
  - ReLU
  - SoftMax
  - Sigmoid

# Algorithm Hyper-Parameters

- Loss function.
  - Cross-entropy for classification
  - L1 or L2 for regression (the labels are continuous)
- Optimization algorithm (and its hyper-parameters, e.g., learning rate).
  - SGD
  - SGD with momentum
  - AdaGrad
  - RMSprop
- Random initialization.
  - Gaussian or uniform?
  - Scaling

# Implement a CNN Using Keras

# 1. Load and Process the MNIST Dataset

## Load data

```
from keras.datasets import mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()

print('Shape of x_train: ' + str(x_train.shape))
print('Shape of x_test: ' + str(x_test.shape))
print('Shape of y_train: ' + str(y_train.shape))
print('Shape of y_test: ' + str(y_test.shape))
```

```
Shape of x_train: (60000, 28, 28)
Shape of x_test: (10000, 28, 28)
Shape of y_train: (60000,)
Shape of y_test: (10000,)
```

# 1. Load and Process the MNIST Dataset

**Reshape:** convert the $60000 \times 28 \times 28$ dataset to a $60000 \times 28 \times 28 \times 1$ tensor.

```
x_train_vec = x_train.reshape((60000, 28, 28, 1)) / 255.0
x_test_vec = x_test.reshape((10000, 28, 28, 1)) / 255.0

print('Shape of x_train_vec is ' + str(x_train_vec.shape))
```

```
Shape of x_train_vec is (60000, 28, 28, 1)
```

# 1. Load and Process the MNIST Dataset

**One-hot encode:** convert the labels (scalars in $\{0, 1, \cdots, 9\}$) to 10-dim vectors.

```
def to_one_hot(labels, dimension=10):
    results = np.zeros((len(labels), dimension))
    for i, label in enumerate(labels):
        results[i, label] = 1.
    return results

y_train_vec = to_one_hot(y_train)
y_test_vec = to_one_hot(y_test)
print('Shape of y_train_vec is ' + str(y_train_vec.shape))
```

```
Shape of y_train_vec is (60000, 10)
```

# 1. Load and Process the MNIST Dataset

Partition the **training** set to **training** and **validation** sets.

```
# Partition to training and validation sets

rand_indices = np.random.permutation(60000)
train_indices = rand_indices[0:50000]
valid_indices = rand_indices[50000:60000]

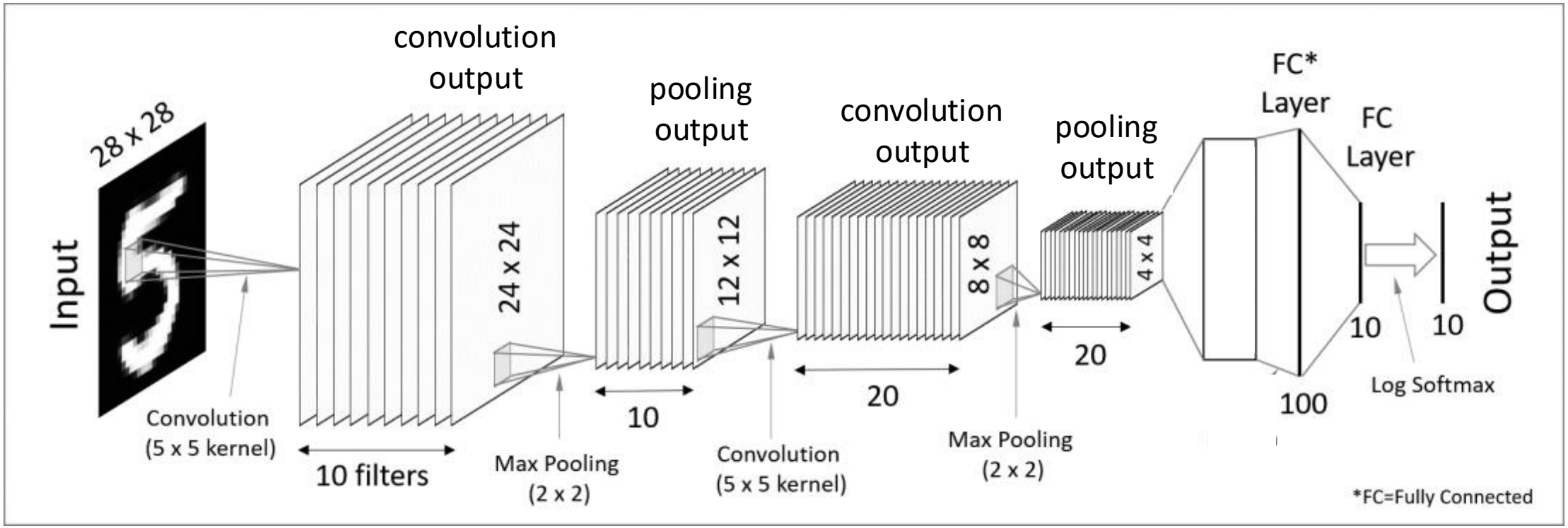
x_valid_vec = x_train_vec[valid_indices, :, :, :]
y_valid_vec = y_train_vec[valid_indices, :]

x_train_vec = x_train_vec[train_indices, :, :, :]
y_train_vec = y_train_vec[train_indices, :]

print('Shape of x_valid_vec: ' + str(x_valid_vec.shape))
print('Shape of y_valid_vec: ' + str(y_valid_vec.shape))
print('Shape of x_train_vec: ' + str(x_train_vec.shape))
print('Shape of y_train_vec: ' + str(y_train_vec.shape))
```

```
Shape of x_valid_vec: (10000, 28, 28, 1)
Shape of y_valid_vec: (10000, 10)
Shape of x_train_vec: (50000, 28, 28, 1)
Shape of y_train_vec: (50000, 10)
```

# 2. Build the CNN



# 2. Build the CNN

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Conv2D(10, (5, 5), activation='relu', input_shape=(28, 28, 1)))
```

Filter number

Filter shape

Default

- Stride: 1
- Zero-padding: no padding

# 2. Build the CNN

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Conv2D(10, (5, 5), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2)))
```

pool_size

Default

- Stride: pool_size

# 2. Build the CNN

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Conv2D(10, (5, 5), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2))) Output: 12 × 12 × 10
model.add(layers.Conv2D(20, (5, 5), activation='relu')) Output: 8 × 8 × 20
model.add(layers.MaxPooling2D((2, 2))) Output: 4 × 4 × 20
```

# 2. Build the CNN

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Conv2D(10, (5, 5), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(20, (5, 5), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
```

Output: $12 \times 12 \times 10$

Output: $8 \times 8 \times 20$

Output: $4 \times 4 \times 20$

Output: 320

# 2. Build the CNN

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Conv2D(10, (5, 5), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(20, (5, 5), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dense(100, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
```

Output: $12 \times 12 \times 10$

Output: $8 \times 8 \times 20$

Output: $4 \times 4 \times 20$

Output: 320

Output: 100

Output: 10

# 2. Build the CNN

```
# print the summary of the model.
model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            (None, 24, 24, 10)        260
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 12, 12, 10)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 8, 8, 20)          5020
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 4, 4, 20)          0
_________________________________________________________________
flatten_1 (Flatten)          (None, 320)               0
_________________________________________________________________
dense_1 (Dense)              (None, 100)               32100
_________________________________________________________________
dense_2 (Dense)              (None, 10)                1010
=================================================================
Total params: 38,390
Trainable params: 38,390
Non-trainable params: 0
_________________________________________________________________
```

# 3. Train the CNN

**Specify:** optimization algorithm, learning rate (LR), loss function, and metric.

```
from keras import optimizers
model.compile(optimizers.RMSprop(lr=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

# 3. Train the CNN

**Specify:** batch size and number of epochs.

```
history = model.fit(x_train_vec, y_train_vec,
                    batch_size=128, epochs=50,
                    validation_data=(x_valid_vec, y_valid_vec))
```

```
Train on 50000 samples, validate on 10000 samples
Epoch 1/50
50000/50000 [==============================] - 12s 239us/step - loss: 1.2455 - acc: 0.7201 - val_loss: 0.4805 - val_acc: 0.8707
Epoch 2/50
50000/50000 [==============================] - 12s 235us/step - loss: 0.3563 - acc: 0.8995 - val_loss: 0.2851 - val_acc: 0.9172
Epoch 3/50
50000/50000 [==============================] - 12s 244us/step - loss: 0.2474 - acc: 0.9271 - val_loss: 0.2230 - val_acc: 0.9365
```

- 
- 
- 

```
Epoch 49/50
50000/50000 [==============================] - 12s 240us/step - loss: 0.0244 - acc: 0.9924 - val_loss: 0.0439 - val_acc: 0.9875
Epoch 50/50
50000/50000 [==============================] - 12s 239us/step - loss: 0.0238 - acc: 0.9927 - val_loss: 0.0446 - val_acc: 0.9881
```

# 4. Examine the Results

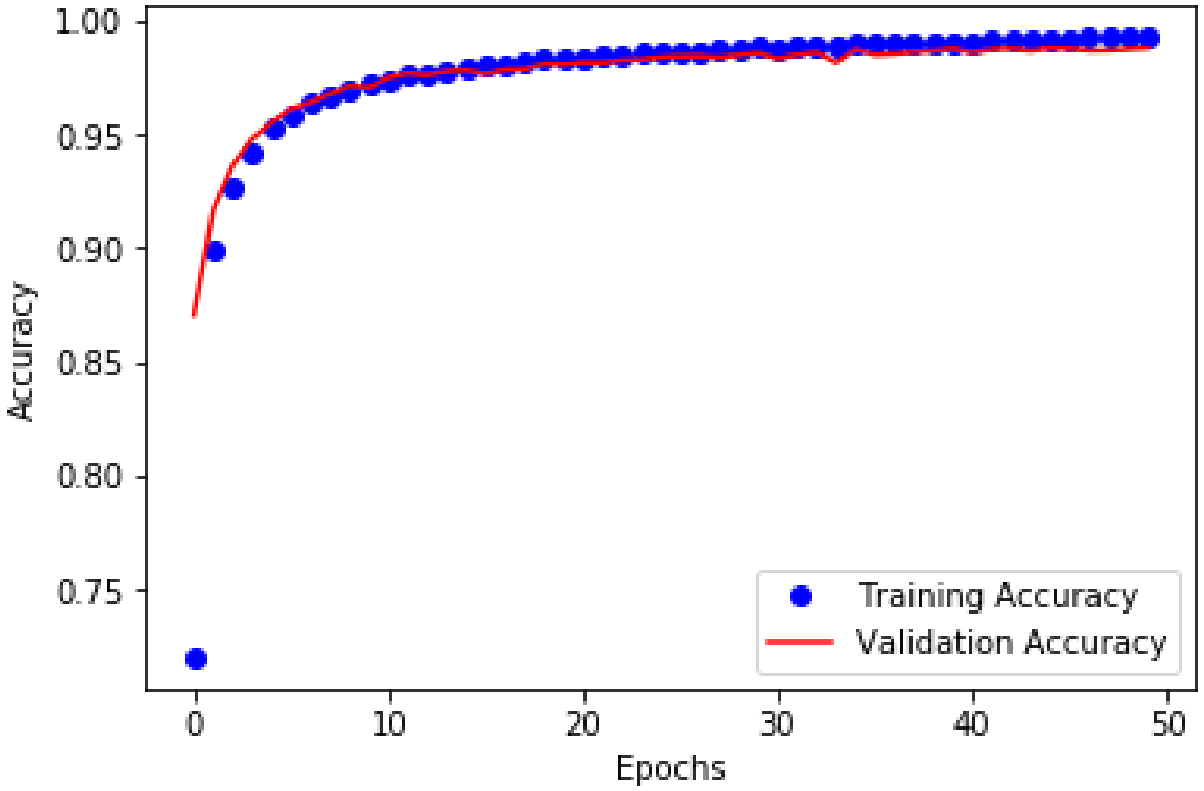
Plot the ***accuracy*** against ***epochs*** (1 epoch = 1 pass over the data).

```
import matplotlib.pyplot as plt
%matplotlib inline

epochs = range(50) # 50 is the number of epochs
train_acc = history.history['acc']
valid_acc = history.history['val_acc']
plt.plot(epochs, train_acc, 'bo', label='Training Accuracy')
plt.plot(epochs, valid_acc, 'r', label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

# 4. Examine the Results

Plot the ***accuracy*** against ***epochs*** (1 epoch = 1 pass over the data).




Why using the validation set?

- Tune the hyper-parameters, e.g., learning rate, filter shape, filter number, etc.
- Check overfitting or underfitting.

# 4. Examine the Results

Evaluate the model on the ***test*** set. (So far, the model has not seen the ***test*** set.)

```
loss_and_acc = model.evaluate(x_test_vec, y_test_vec)
print('loss = ' + str(loss_and_acc[0]))
print('accuracy = ' + str(loss_and_acc[1]))
```

```
10000/10000 [==============================] - 1s 127us/step
loss = 0.035708001047297 15
accuracy = 0.9883
```

# 4. Examine the Results

- Training accuracy: 99.3%
- Validation accuracy: 98.8%
- Test accuracy: 98.8%

# Summary

# 1. Convolutional Layer

- Key concepts:
  - filter (kernel)
  - patch
  - output (feature map)
  - zero-padding
  - stride

# 2. Convolutional Neural Network

- Convolutional layers.
  - Filter number.
  - Filter shape.
  - Stride (default: one).
  - Zero-padding (default: no padding).

# 2. Convolutional Neural Network

- Convolutional layers.
  - Filter number.
  - Filter shape.
  - Stride (default: one).
  - Zero-padding (default: no padding).
- Activation functions.
- Pooling layers.
  - MaxPool, AveragePool, etc.
  - Pool size.
  - Pool stride (default: pool size).

# 3. Build and Train CNN using Keras

- Build a network.
  - Add Conv, Pool, FC layers.
- Compile the model.
  - Specify optimization algorithm, loss function, and metrics.
- Fit the model on training data.

# Thank you!